

Technical Project Review

System Overhaul & Scalability Initiative

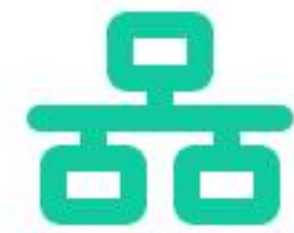
ENGINEERING TEAM REVIEW

Review Pillars



1. Problem

Defining the core technical bottlenecks, system limitations, and the business impact driving this initiative.



2. Architecture

High-level system design, component interactions, and infrastructure topology to support scale.



3. Solution (TDD)

Detailed technical design, API contracts, data models, and specific implementation patterns chosen.



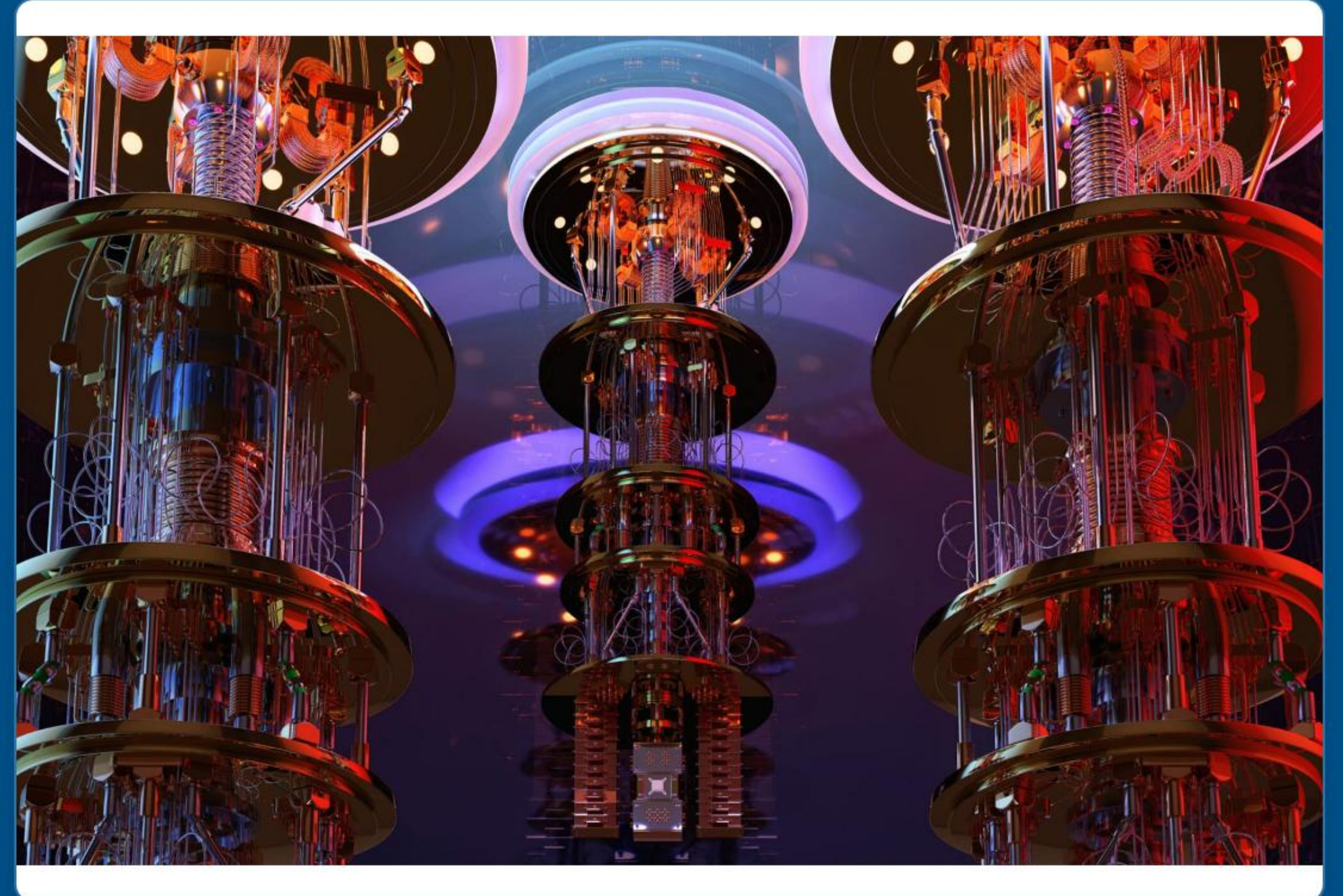
4. Test Cases

Quality assurance strategy, coverage metrics, and critical test scenarios ensuring system integrity.

1. The Problem Statement

Our current monolithic architecture is reaching its operational limits, impacting user experience and development velocity.

- **High Latency:** P99 API response times exceed 2.5 seconds during peak traffic hours, causing timeout errors.
- **Data Inconsistency:** Legacy batch processing leads to stale reads and occasional race conditions in inventory states.
- **Deployment Friction:** Tightly coupled components require massive, risky deployment windows resulting in infrequent releases.
- **Scaling Inefficiency:** Inability to horizontally scale isolated high-traffic services independently.



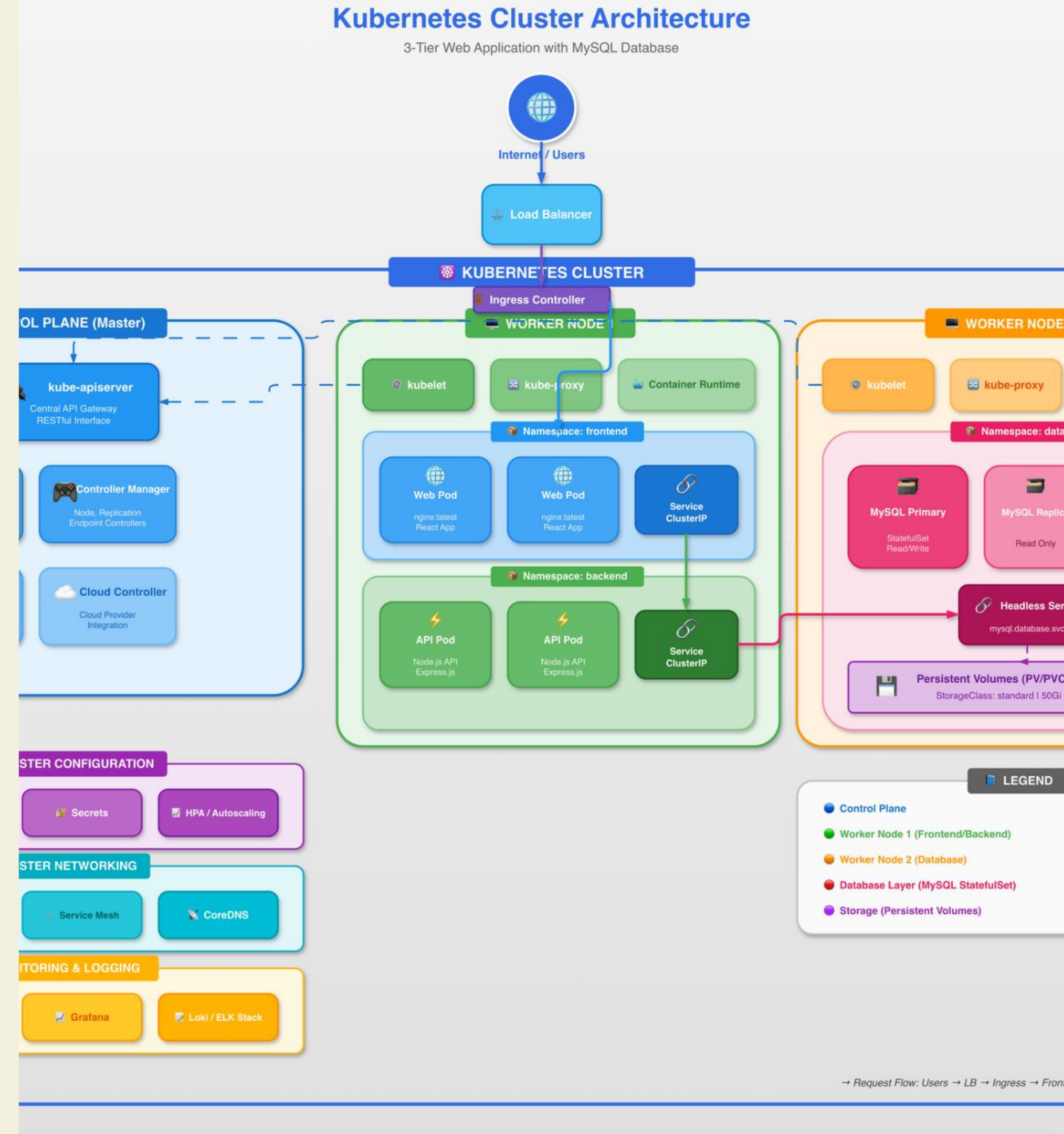
2. Target Architecture

Microservices Transition

We are migrating to an event-driven, domain-centric microservices architecture to enforce hard boundaries and enable independent scaling.

Key architectural shifts:

- Implementation of an API Gateway for request routing and rate limiting.
- Decomposition of the core engine into 'Order', 'Inventory', and 'User' bounded contexts.
- Introduction of Apache Kafka as a central event bus to eliminate synchronous cross-service calls.



Architecture Components

Data Tier

Moving away from a single shared relational database to polyglot persistence.

- **PostgreSQL:** Primary ACID store for transactional domains (Orders, Payments).
- **MongoDB:** Document store for dynamic product catalogs and user profiles.
- **Redis:** Distributed caching layer for session management and API response caching.

Infrastructure

Fully containerized workloads managed via modern orchestration.

- **Compute:** Kubernetes (EKS) clusters configured with horizontal pod autoscalers (HPA).
- **CI/CD:** GitHub Actions pipeline enforcing zero-downtime rolling deployments.
- **Observability:** Datadog for distributed tracing, log aggregation, and APM.

3. Solution Design (TDD Excerpts)



API Contracts & GraphQL

Moving from REST to GraphQL for the client-facing gateway. This allows frontend clients to specify exact data requirements, solving the over-fetching issue currently saturating mobile network bandwidth.



Idempotency in Payments

To prevent duplicate charges during network failures, the Technical Design Document enforces idempotency keys for all mutating state operations, verified via Redis before database commits.



Security Posture & Auth

Implementing OAuth2.0 with short-lived JWTs and refresh token rotation. Microservices will validate tokens via JWKS endpoints rather than proxying every request to the central Auth service.

Design Trade-offs Assessed



Eventual Consistency

Chosen over strict ACID across services to guarantee high availability and low latency during inventory updates.



Managed Services

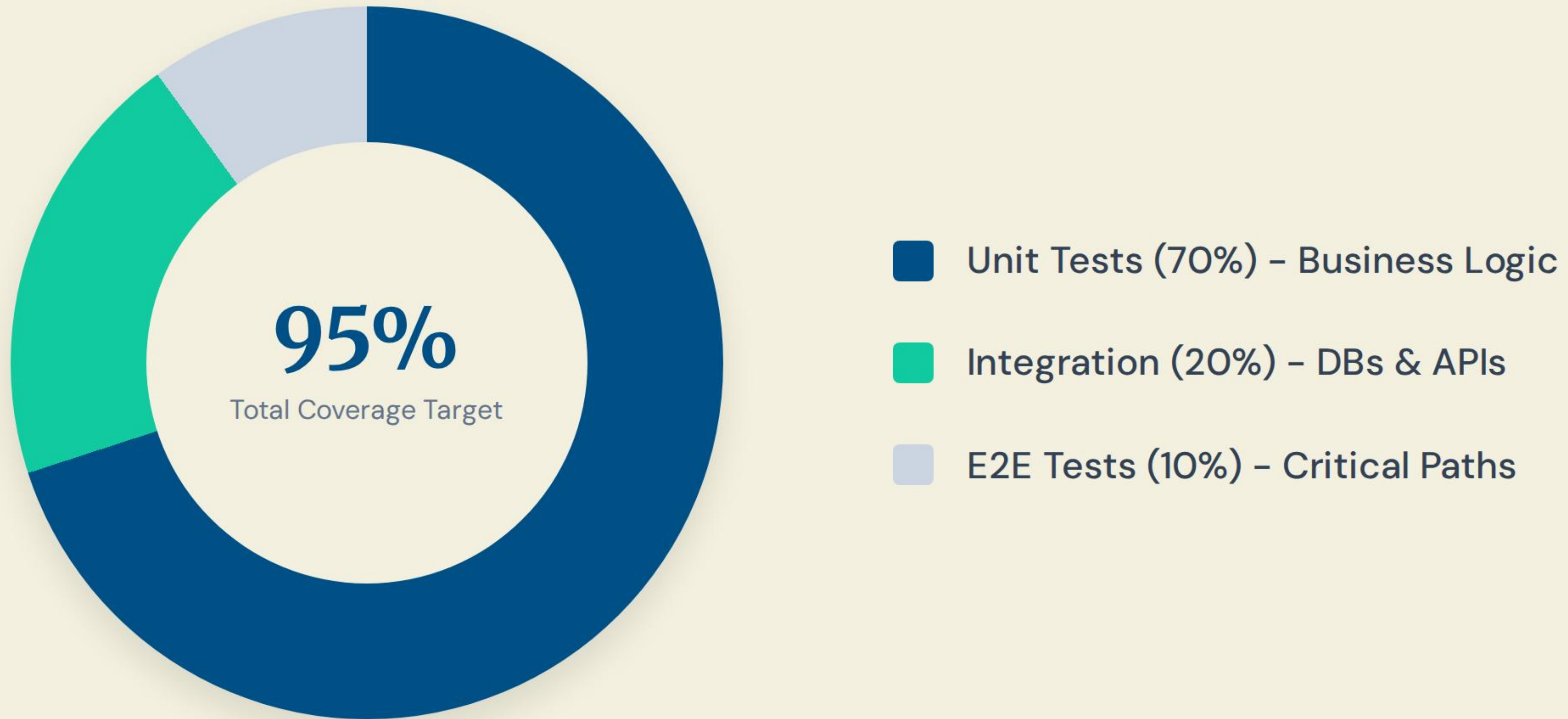
Opting for managed Kafka (Confluent) despite higher OPEX to drastically reduce DevOps overhead and maintenance time.

 Abstract image representing cyber security

Network Boundaries

Implementing strict mTLS between internal services adds minor latency but satisfies stringent compliance and security audits.

4. Automated Test Strategy



Following the testing pyramid: Heavy emphasis on fast, deterministic unit tests. E2E tests strictly reserved for critical user journeys (e.g., checkout flow).

Critical Test Scenarios

Domain / Component	Test Scenario	Validation Focus	Priority
API Gateway	Traffic Spike / Rate Limiting	Ensure 429 status is returned accurately; no backend service degradation.	Critical
Payment Service	Network timeout during commit	Idempotency key prevents double charging upon automatic retry.	Critical
Event Bus (Kafka)	Broker node failure simulation	Consumers successfully rebalance; zero message loss guaranteed.	High
Inventory DB	Concurrent checkout requests	Optimistic locking prevents overselling of limited stock items.	High
Auth Service	Token rotation boundary	Seamless transition without forcing user re-authentication on frontend.	Medium



Review & Feedback

Opening the floor for technical critique, architecture validation, and alignment.

Image Sources



<https://postquantum.com/wp-content/uploads/2026/02/Quantum-Open-Architecture-QOA-scaled.jpg>

Source: postquantum.com



<https://robotpaper.ai/content/images/2026/02/image-43.png>

Source: robotpaper.ai



https://axial.acs.org/_next/image?url=https%3A%2F%2Fcdn.sanity.io%2Fimages%2F%2Fproduction%2Fd4e7e41bd56ea4d5d186edb2bc08aeaba6de2b77-1200x628.jpg&w=2560&q=80

Source: axial.acs.org



<https://cdn.shopify.com/s/files/1/0871/2798/2406/files/11af4267-8396-465a-8337-5e1714bf0ea3.jpg?v=1776343214>

Source: rossettiart.com



https://lh3.googleusercontent.com/QntNnmnSqGie9nfFNL-NWI5upJIC6kV8JugTVny1l_knkK-mGdGcF1hMPXH09oAEG19eNzQOXUTweVOxWsgdIVdrq91YtcxxOgkpv8mzYnSzoF&x=w1440-h810-n-nu

Source: deepmind.google